README

Task #80 evidence — docs_chain sync/verify disagreement on codegraph-status

Revision: 1 **Last modified:** 2026-08-18T20:20:00Z

Symptom

`docs_chain sync --all` reported `codegraph-status` IN-SYNC (no changes) while `docs_chain verify --all` reported `STALE: [status_docx]` for the SAME state, on every run, permanently.

Root cause

`docs/codegraph/Status.docx` (a git-ignored, locally-generated build derivative — see `.gitignore` line `*.docx`) had been overwritten out-of-band by a plain `pandoc -f markdown -t docx` invocation (traced to `scripts/generate_markdown_exports.sh`'s mtime-based, docs_chain-unaware DOCX export step — see "Concerns" below), while `docs/codegraph/Status.md` stayed untouched. Confirmed via `docProps/core.xml` inside the two docx zips:

- The drifted on-disk docx: `<dc:title></dc:title>` (empty) + `<dcterms:created>2026-08-18T14:53:05Z</dcterms:created>` (real wall-clock time).
- A correct `docs_chain`-produced docx (via its `PandocMarkdownToDOCX` builtin): `<dc:title>Status</dc:title>` + `<dcterms:created>2000-01-01T00:00:00Z</dcterms:created>` (the pinned `SOURCE_DATE_EPOCH`).

No `docs_chain` invocation could have produced the drifted bytes — proving the file was written by a foreign tool.

Engine bug (constitution/submodules/docs_chain, `internal/graph/recompute.go`)

`graph.Recompute`'s Step 1 hashes EVERY node (source AND derived) against its stored baseline to build the dirty set — this correctly detected the docx's self-drift. But Step 3's topo-walk candidacy for a derive-from TARGET consulted ONLY whether its SOURCE was dirty, never whether the target's OWN on-disk bytes had drifted from its own baseline — so the drifted docx was never regenerated. `orchestrator.Run` then called `g.CommitHashes` unconditionally (even on the in-sync fast path), which silently adopted the drifted on-disk hash as the new "correct" baseline. From that moment, `sync` reported "in-sync" forever (self-consistent with the now-corrupted baseline) while `verify` — which never consults the baseline and always freshly re-derives every target from its live sources — permanently reported it stale.

Fix

`constitution/submodules/docs_chain` commit `8e0abe0` (two parts, both required):

1. Recompute candidacy for a derive-from target now ALSO fires on the target's own self-drift (`dirty[id]` from Step 1), not only a dirty source.
2. The early-cutoff comparison for a newly-candidate target is now against its CURRENT on-disk hash (captured in Step 1), never the OLD stored baseline. For a normal source-triggered candidate the two are identical by construction (no behaviour change); for a self-drift candidate the old baseline still equals the CORRECT content, so comparing against it would wrongly early-cutoff-prune the healing write.

RED-first regression guard: `internal/runner/sync_derived_self_drift_test.go` drives the exact sequence through the real runner (sync → out-of-band corruption → sync → verify). Confirmed FAIL pre-fix (`bash-style paste below`), PASS post-fix. `go test -race -count=1 ./...` green across all 8 packages, both before and after (the fix introduces zero regressions).

Immediate mitigation (this project)

Deleted the drifted local `docs/codegraph/Status.docx` (safe — git-ignored build derivative, never committed) and re-ran `docs_chain sync --all` under the fixed engine, which regenerated it correctly. See `verify_before.txt`, `sync_output.txt`, `verify_after.txt`, `challenge_output.txt`, and `docprops_diff.txt` in this directory for the pasted terminal evidence.

Pointer cascade

- `constitution/submodules/docs_chain` → `8e0abe0` (fix landed, pushed to `github.com:vasic-digital/docs_chain.git`, and mirrored to `gitflic.ru/gitlab.com` via `origin`'s multi-push).
- `constitution` (this project's submodule) → `dd3b888` (bumps the docs_chain pointer, pushed to `github/gitflic/gitlab` — `gitverse` rejected the push with a PRE-EXISTING, unrelated permission error, see Concerns).
- boba `constitution` submodule pointer → `7077107` (this commit), pushed to `github.com:milos85vasic/Boba-Base.git`.

Concerns / follow-ups (out of this task's scope)

1. **True root cause of the original corruption is `scripts/generate_markdown_exports.sh`** (explicitly out of scope — "Do NOT touch: … scripts/"). Its DOCX step runs a raw `pandoc -f markdown -t docx -o "$docx" "$md"` with NO `SOURCE_DATE_EPOCH` pinning and NO `--metadata title=...`, gated only by mtime (`$md -nt $docx`) — the exact §11.4.86 "mtime not content-hash" anti-pattern the constitution forbids. Per §11.4.106(A), any doc owned by a registered `docs_chain` context should be retired from this legacy script's scope so it can never again clobber a docs_chain-owned derived export. Recommend a follow-up workable item to either (a) skip any `.md` file that has a matching `.docs_chain/contexts/*.yaml` node entry, or (b) retire the script's DOCX/HTML/PDF steps entirely in favour of `docs_chain sync --all`.
2. The `docs_chain` submodule's `.gitignored` `.docs_chain/state.json` already held the drifted hash as its "baseline" for `status_docx` BEFORE this session started — meaning some earlier `docs_chain sync` invocation (using a pre-fix, already-stale-relative-to-today binary) already absorbed the corruption into the baseline. The new engine fix prevents this class of corruption from happening AGAIN; it cannot retroactively detect corruption that was already fully "settled" into the baseline before the fix was deployed (there is no more hash mismatch to detect once both sides agree on the wrong content) — hence the explicit delete-and-regenerate mitigation step above was required.
3. `gitverse.ru:helixdevelopment/constitution.git` push (a `constitution` submodule remote) failed with `"Push to create is not enabled for organizations"`; the SAME remote also failed at `git fetch` moments earlier with `"Cannot find repository"`. This is a pre-existing, unrelated remote-configuration issue (confirmed not caused by this session's changes) — flagging for operator awareness, not fixed here (out of scope).

4. A large volume of concurrent, unrelated file churn (other subagents' in-flight work — `docs/qa/BOB-075/concurrency_evidence/**`, several `docs/incidents/*`, `docs/proposals/*`, various `docs/scripts/*.html` /`.pdf` exports, `submodules/challenges`, `submodules/helixqa`, `scripts/commit-push-all.sh`) was present in the boba working tree throughout this session (multi-track parallel development per §11.4.58/.103/.176/.187/.192). Per §11.4.84 working-tree-quiescence discipline, every commit in this session staged ONLY the single intended file (`constitution` in boba; `internal/graph/recompute.go` + the new test file in docs_chain; `submodules/docs_chain` in constitution) — never `git add -A` — so none of that concurrent work was touched or swept in.